

RD2ESC: 多智能体嵌入式代码生成框架

谭舒孺¹ 肖宏彬¹ 李智² 谢晓兰³ 武天昊³ 汤飞⁴

¹(教育部教育区块链与智能技术重点实验室(广西师范大学) 广西桂林 541004)

²(广西师范大学计算机科学与工程学院 广西桂林 541004)

³(桂林理工大学计算机科学与工程学院 广西桂林 541004)

⁴(华为技术有限公司 广东深圳 518000)

(1392965146@qq.com)

RD2ESC: Multi-Agent Embedded Code Generation Framework

Tan Shuru¹, Xiao Hongbin¹, Li Zhi², Xie Xiaolan³, Wu Tianhao³, and Tang Fei⁴

¹(Key Lab of Education Blockchain and Intelligent Technology (Guangxi Normal University), Ministry of Education, Guilin, Guangxi 541004)

²(College of Computer Science and Engineering, Guangxi Normal University, Guilin, Guangxi 541004)

³(College of Computer Science and Engineering, Guilin University of Technology, Guilin, Guangxi 541004)

⁴(Huawei Technologies Co., Ltd., Shenzhen, Guangdong 518000)

Abstract Large language models (LLM) are increasingly being applied in software engineering, but current automated code generation research primarily focuses on general-purpose functional code, lacking effective solutions for the specific requirements of embedded systems. We propose RD2ESC (requirements documents to embedded system code), a prompt-based fine-tuning method that enables LLMs to understand the complex relationships between embedded code and requirements documents, and constructs a multi-agent collaborative code generation framework capable of rapidly generating high-quality embedded code using requirements documents and reference code. Experimental results demonstrate that RD2ESC improves the Pass@1 metric from 0.15 to 0.71 compared with the GPT-4o baseline model, achieving a test pass rate of 0.75 and compilation pass rate of 0.96. Sensitivity analysis reveals that the method exhibits certain dependency on reference code quality, with Pass@1 declining from 0.68 to 0.47 under 10%~50% perturbation conditions and dropping to 0.25 without reference code, while still maintaining basic code generation capabilities. Ablation experiments confirm the synergistic effects among multi-agents, with the complete system demonstrating significant performance improvements compared with individual components. This research provides an effective technical framework for embedded code automatic generation, substantially enhancing embedded system development efficiency.

Key words embedded system code generation; multi-agent collaboration; large language models; prompt engineering; compiling feedback optimization

摘要 大语言模型(LLM)在软件工程中的应用日益广泛,但目前自动化代码生成研究主要集中于通用功能代码,缺乏针对嵌入式系统特殊需求的有效解决方案。提出了RD2ESC(requirements documents to embedded system code)方法,通过基于提示词的微调技术使LLM能够理解嵌入式代码与需求文档之间的

收稿日期: 2025-10-30; 修回日期: 2025-12-30

基金项目: 国家自然科学基金项目(62362006); 广西科技计划项目(桂科AB24010343)

This work was supported by the National Natural Science Foundation of China (62362006) and the Guangxi Science and Technology Program (GuikeAB24010343).

通信作者: 李智(zhili@gxnu.edu.cn)

复杂关系,并构建了多智能体协同的代码生成框架,能够利用需求文档和参考代码快速生成高质量的嵌入式代码。实验结果表明,RD2ESC 相比 GPT-4o 基线模型在 Pass@1 指标上从 0.15 提升至 0.71,测试通过率达到 0.75,编译通过率达到 0.95;敏感性分析显示该方法对参考代码质量存在一定依赖性,在 10%~50% 扰动条件下 Pass@1 从 0.68 降至 0.47,完全无参考代码时降至 0.25,但仍保持基础代码生成能力;消融实验证实了多智能体间的协同效应,完整系统相比单一组件展现出显著的性能提升。该研究为嵌入式代码自动生成提供了有效的技术框架,提升了嵌入式系统开发效率。

关键词 嵌入式系统代码生成;多智能体协作;大语言模型;提示工程;编译反馈优化

中图法分类号 TP311

DOI: 10.7544/issn1000-1239.202550663

CSTR: 32373.14.issn1000-1239.202550663

在嵌入式系统开发领域,如何高效、准确地将需求文档转化为可执行代码,是影响项目成败的关键环节。嵌入式代码生成的核心任务,是将自然语言或结构化需求描述自动转化为能够在特定硬件平台上运行的高质量代码。高效的自动化代码生成能够显著加速需求验证,降低人工出错率,提升产品开发的整体效率和可靠性。提升嵌入式代码生成效率和质量的关键,在于实现对复杂需求的准确解析与映射,充分复用历史参考代码和工程知识,自动适配不同硬件平台和编程范式,并借助自动化反馈机制不断优化生成结果。理想的自动化工具应能够理解自然语言需求,智能检索与融合相关代码片段,及时响应需求变更,并生成可编译、可测试的高质量初始实现,从而减少人工适配和调试的工作量。

大语言模型(large language model, LLM)在代码生成方面取得了显著进展,为软件自动化带来了新的可能性。然而,尽管 LLM 在通用编程领域表现优异,当前针对嵌入式系统这一具有强硬件依赖、参考数据稀缺、验证方法复杂、编码约束严格以及高实时性要求等特点的专业领域,其代码生成研究仍面临独特挑战。现有的 LLM 代码生成研究主要集中在更易实现的简单需求到功能代码的转换上,例如 HumanEval 数据集中的测试集合^[1]。虽然模型驱动开发(model-driven development, MDD)方法在嵌入式系统领域已有广泛应用,如 MATLAB/Simulink 和 Rhapsody 等工具能够从形式化模型生成代码^[2],但 these 方法需要开发者掌握特定的建模语言,且生成的代码往往需要大量手工优化才能满足嵌入式系统的严格资源约束。相比之下,基于 LLM 的方法能够直接理解自然语言需求,提供更灵活的代码生成方案,但如何确保生成代码的质量和可靠性仍是一个开放性问题。

正是基于这一挑战与机遇,本研究展开了相关

探索。本文研究了利用 LLM 从需求文档自动生成可执行嵌入式代码的方法。我们的核心技术创新在于构建了一个基于多智能体协作的嵌入式代码生成框架,该框架通过思维链(chain-of-thought, CoT)推理和提示词工程技术构建 4 个专门化智能体,实现协同工作来提升代码生成的质量和可靠性。具体而言,我们设计了以下 4 个功能互补的智能体:参考代码生成智能体负责学习和理解 3 种不同编程范式(面向过程、模块化和结构化)的嵌入式代码实现模式,生成 3 种不同风格的参考代码;需求驱动代码生成智能体基于输入的需求文档生成符合嵌入式系统特点的初始代码实现;编译反馈智能体通过集成编译器反馈机制,自动识别和定位生成代码中的语法错误、类型不匹配等基础问题并给出编译反馈信息;代码优化建议智能体基于 3 种不同风格的参考代码分析生成的初始代码中的问题并提供针对性的修正策略。

与以往基于 LLM 的需求到代码生成研究^[3-4]相比,我们的工作实现了以下 3 项技术创新:

1)反思驱动的多智能体协作架构。将 LLM 的代码生成能力与反思驱动的多智能体协作机制相结合,通过需求驱动代码生成智能体、参考代码生成智能体、编译反馈智能体和代码优化建议智能体的分工协作,构建了从需求理解到高质量代码输出的端到端自动化流程,实现了“感知—诊断—修复”的闭环优化机制。

2)编译反馈驱动的迭代优化策略。通过引入编译反馈机制和多轮迭代优化,系统能够智能识别语法错误、逻辑缺陷和功能不完整等问题,并基于精确的错误诊断进行针对性修复。实验表明,该机制使 Pass@1 性能相比直接使用 LLM 生成代码提升了数倍,编译通过率达到 95%,超越了现有的单轮生成方法。

3)分层质量保障与协同放大效应。本文框架使

用了分离编译正确性与功能正确性的质量保障机制,消融实验证实了智能体间的非线性协同效应,完整系统相比单一组件展现出显著的性能放大,为嵌入式代码生成任务提供了新的参考。

1 背景与相关工作

1.1 嵌入式代码生成

嵌入式软件开发的历史可以追溯到 20 世纪 70 年代,早期开发主要依赖手工编写汇编代码,开发者需要直接与硬件交互。随着嵌入式系统应用场景的不断扩展和复杂性的显著增加,开发方法也经历了持续的演进:从早期主要遵循传统“需求—设计—编码—测试”流程、依靠手动将自然语言需求转化为汇编或 C 语言代码以及 1990 年代以来模型驱动开发(model-driven development, MDD)方法的广泛应用和持续发展^[5-6]。UML、SysML 等标准化建模语言以及 IBM Rhapsody、MATLAB/Simulink、AUTOSAR 工具链等商业工具在汽车电子、航空航天等关键领域取得了显著成功,实现了从模型到代码的自动化转换,并在安全关键系统的开发中建立了成熟的工程实践。然而,尽管 MDD 方法在许多工业领域已经相当成熟,但在快速原型开发和需求频繁变更的场景中,传统 MDD 流程可能显得相对繁重,且自然语言需求到形式化模型的转换仍然主要依赖人工完成。正是在这一背景下,基于 LLM 的代码生成方法为嵌入式开发提供了一种补充性的技术路径,特别是在需求验证和快速原型开发阶段,为传统开发流程提供了有益的技术补充。

在工业应用中,嵌入式系统开发确实面临着独特的挑战和严格的要求。汽车电子控制单元(ECU)、工业自动化控制器以及医疗设备等领域,不仅要求代码在功能上是正确的,还必须满足严格的实时性、可靠性和安全性要求。为应对这些挑战,业界已建立了相对成熟的开发体系: AUTOSAR(汽车开放系统架构)^[7-8]等规范为嵌入式软件架构提供了标准化框架,配合相应的工具链和开发流程,已在关键领域得到广泛应用并积累了丰富的工程经验。然而,在项目早期阶段,从自然语言需求文档到符合这些标准的初始代码实现之间仍存在一定的转换鸿沟。在工业实践中,需求理解和初步验证阶段往往需要经历多轮迭代:工程师需要将业务需求转化为技术规格说明,再进一步实现为可编译的代码原型,然后通过硬件部署和系统测试来验证需求的正确性和完整

性。这一早期验证过程,特别是在需求频繁变更或探索性项目中,可能成为影响整体开发效率的瓶颈。因此,如何在保持工业级质量标准的前提下,加速从需求到可验证代码原型的转换过程,成为值得探索的技术问题。

1.2 系统模型基于 LLM 的代码生成

近年来, LLM^[9-11]发展迅速,在代码生成领域取得了显著成果^[12-16]。Copilot^[17]作为基于 LLM 的知名代码生成工具,已在通用软件开发中展现出其实用性,其生成的代码被超过 30% 的用户所接受^[18]。现有的代码生成模型可以分为两大范式:编码器-解码器模型和仅解码器模型。编码器-解码器模型如 PLBART^[19]、CodeT5^[20]和 AlphaCode^[21],会将输入文本编码为上下文嵌入向量,再解码为代码解决方案。仅解码器模型如 GPT 系列^[22-23]、PolyCoder^[24]和 InCoder^[16],则采用下一个 token 预测目标,从左到右生成代码。其中,OpenAI 开发的 ChatGPT^[24]和 GPT-4^[25]代表了最先进的 LLM,展现出更强的理解与推理能力,以及高质量文本生成能力。为了进一步提升代码生成质量,研究者们提出了多种改进策略: ROCODE^[26]集成了回溯机制和程序分析技术来提高代码生成的正确性; MBR-EXEC^[27]基于执行结果的最小贝叶斯风险框架来选择最优代码版本; MGD^[28]采用监控器引导解码策略,通过静态分析为模型提供全局上下文信息,减少代码生成中的幻觉现象。

鉴于训练或微调这些 LLM 的成本极高,许多研究聚焦于在极少甚至无需微调的情况下提升其代码生成性能。提示学习(Prompt learning)成为实现这一目标的关键技术之一^[29-35]。CoT^[33]作为一种创新的提示工程方法,引导 LLM 生成中间推理步骤,从而得到最终答案。在复杂推理任务中, CoT 表现出色^[35-36],因此也被应用于代码生成^[35-38]。受到 CoT 的启发, Li 等人^[4]提出了结构化 CoT(SCOT),该方法明确引入代码结构,并指导 LLM 以程序化结构生成中间推理步骤。Jiang 等人^[14]则提出了一种自我规划方法,通过少样本演示引导 LLM 理解代码规划,并为给定需求编写相应的代码计划。

近年来,多智能体协作方法在代码生成领域展现出显著优势。ChatDev^[39]通过构建包含 CEO、CTO、程序员、测试员等 7 个专业角色的智能体团队,利用链式通信机制实现从需求分析到代码实现的全流程协作; AgentCoder^[40]则采用更精简的三智能体架构(程序员智能体、测试设计师智能体、测试执行智能体),通过独立的测试生成和执行反馈机制。此外,

自一致性增强方法如 Self-Edit 通过执行测试用例并基于错误信息迭代优化代码,进一步提升了代码生成的可靠性。

本文的方法同样采用提示工程技术优化 LLM,实现从需求到代码的转化。与传统的代码生成方法不同,我们的核心创新在于构建了一个适配嵌入式系统特点的多智能体协作框架 RE2ESC(Requirements documents to embedded system code),通过 CoT 推理和提示词工程技术将复杂的代码生成任务分解为多个专门化的子任务。该框架包含参考代码生成智能体、需求驱动代码生成智能体、编译反馈智能体和代码优化建议智能体 4 个功能模块。其中编译反馈智能体直接集成 Keilu Vision5 C51 编译器,实现了面向真实部署环境的代码验证机制。在嵌入式系统适配方向,提示词工程中融入了编译器约束(如内存使用约束;避免使用动态内存分配(malloc/free)实时性约束;主循环执行时间控制在 10 ms 以内,避免使用阻塞式延时函数等)和硬件特性考虑,生成的代码直接输出可烧录的 .hex 文件,实现从需求到可部署代码的端到端转换。

同时相较于依赖模拟测试环境的通用多智能体方法^[39-41],我们的架构通过编译器直接反馈和智能体通过参考代码提供建议的迭代优化机制,有效解决了硬件相关功能(LCD 显示、传感器交互等)在纯软件环境中验证复杂的问题。这种策略显著减少了从需求到可测试代码的生成时间,通过多智能体协作生成编译通过率更高、功能完整性更好的初始代码,开发者可以更快地进入实际硬件测试阶段。实验结果表明,相比直接使用目前的多智能体开发方法,我们的方法将首次通过率(Pass@1)提升了 28%,平均测试用例通过率提升了 20%。

需要明确的是,本文的方法是将 LLM 技术引入嵌入式代码生成领域的探索性研究,旨在为现有嵌入式软件开发流程提供技术补充,而非为对目前现有成熟工业实践的替代。该多智能体协作框架通过提供更高质量的初始代码来减少早期需求验证阶段的迭代次数,从而在保持现有质量保证体系的前提下,提高开发流程的特定环节效率。

2 基于多智能体协作的嵌入式代码生成方法

在实际的嵌入式软件开发过程中,开发者通常遵循一套成熟的工作流程:首先收集和分析已有的代码模板和参考实现,构建项目的技术知识基础;然

后基于需求文档和参考代码编写初始版本的代码;接着通过编译器验证代码的语法正确性和依赖关系;最后根据编译错误和警告信息对代码进行调试和修正,直到获得可正常编译运行的最终代码。本文提出的多智能体协同框架正是基于这一实际开发流程设计的。通过将传统开发过程中的关键环节抽象为专业化的智能体,每个智能体模拟开发者在特定阶段的专业技能和决策过程,从而实现了对人类开发经验的有效建模和自动化。具体而言,首先基于用户给予的需求文档通过需求驱动代码生成智能体与编译反馈智能体生成初始的代码,随后基于用户给予的参考代码与参考代码需求文档生成 3 种不同风格的参考代码,之后代码优化建议智能体根据 3 份参考代码与需求文档纠正初始代码中的问题,并结合编译反馈智能体纠正生成代码中的编译错误以达到生成高质量代码的目标。这种设计不仅保证了代码生成过程的合理性和可靠性,还充分利用了多智能体协作系统的优势,提高了代码生成的效率和质量。

在本节,将介绍通过多智能体协同技术构建的嵌入式代码生成方法,如图 1 展示了多智能体协作框架的 4 个主要组成部分,包含参考代码生成智能体、编译反馈智能体、需求驱动代码生成智能体与代码优化建议智能体,各智能体的工作流程将下面具体介绍。

2.1 编译反馈智能体

编译反馈智能体 A_{compile} 的构建旨在减少代码生成后模型因为语法错误导致的代码生成质量问题。

其输入包含待编译代码 c ,输出结果为结构化编译反馈信息 F 与根据编译代码 c 生成的可用于进行嵌入式系统测试的 .hex 文件。形式化表达如式(1)所示。

$$A_{\text{compile}}(c) \rightarrow (F, \text{hex})。 \quad (1)$$

结构化编译反馈 $F=(\text{status}, \text{errors}, \text{warnings}, \text{suggestions})$ 如式(2)所示。其中 status 表示代码编译结果,包含成功 SUCCESS 和失败 FAILURE 两种状态; errors 为编译错误信息集合,包含错误类型 type 、错误所在行数 line 、错误具体信息内容 msg ; warnings 表示警告信息集合,包含警告类型 category 、警告描述 desc , i 与 j 表示错误信息与警告信息索引。

$$\begin{aligned} \text{status} &\in \{\text{SUCCESS}, \text{FAILURE}\}, \\ \text{errors} &= \{(\text{type}_i, \text{line}_i, \text{msg}_i)\}_{i=1}^k, \\ \text{warnings} &= \{(\text{category}_j, \text{desc}_j)\}_{j=1}^l, \\ \text{suggestions} &。 \end{aligned} \quad (2)$$

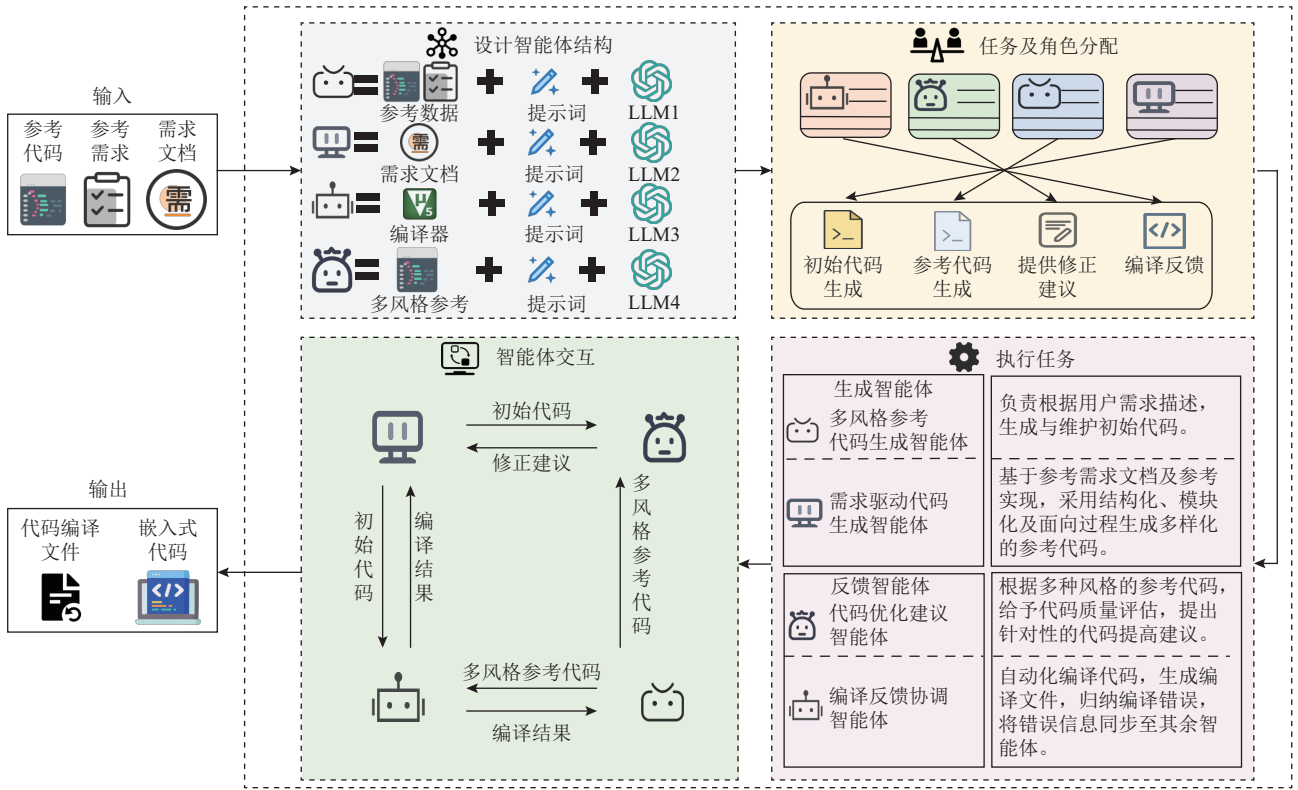


Fig. 1 Multi-agent embedded code generation framework

图1 多智能体嵌入式代码生成框架

在本文中该智能体通过 LLM 与可调用的编译器 Keilu Vision5 C51 实现。

2.2 需求驱动代码生成智能体

需求驱动代码生成智能体的目标为根据提供的需求文档并结合编译反馈智能体迭代修改生成一份编译成功的初始嵌入式系统代码,同时负责后续代码问题的维护工作。在代码生成任务中,我们发现代码生成智能体经常遇到 3 类典型问题:使用编译器不允许的参数名、错误地假设某些功能已被实现,以及给出不完整的代码实现。为了解决这些问题,我们提出了双重优化策略,以在第 1 轮代码生成过程中提升代码质量。

首先,在构建需求驱动代码生成智能体的提示词的设计阶段,我们采用了 CoT 技术^[31]引导模型进行结构化思考。该技术将复杂任务分解为一系列推理步骤,使模型能够系统地分析问题需求、设计解决方案并实现代码。其次,我们在提示词中明确加入了约束条件,如“避免使用保留关键字作为标识符”和“为每个功能提供完整实现”等约束,减少了常见错误。根据认知负荷理论,这一方法降低了模型的认知负担,使其能更专注于代码质量,而不仅仅是完成任务。所设计的提示词如图 2 所示。

你将扮演一名具有丰富经验的嵌入式系统设计师,结合给出的需求文档,通过代码实现一个嵌入式系统。下面2个引号之间是一份需求文档,请根据这些信息先生成构建系统需要的步骤与完整的C语言代码,每个功能部分逐步实现。以下为生成代码时的注意项:

1. 禁止使用data作为参数。
2. 代码编译器为KeiluVision5。
3. 假设不存在任何已知驱动文件。
4. 设计的代码需要实现全部功能。
5. 只输出完整的C语言代码,不要其他内容。
6. 要求格式为:
...“你的思路”...
...“code你的代码”...
其中代码为完整的C语言代码。...

###需求文档内容:
需求文档: 基于8051单片机的实时时钟 (RTC) 和LCD显示系统 ...

Fig. 2 Key prompts for initial code agent

图2 初始代码智能体关键提示词

需求驱动代码生成智能体 A_{gen} 的输入为待生成代码的需求文档 RD ,输出为编译验证后的初始代码 c_{initial} 。形式化表达如式(3)所示。

$$A_{\text{gen}}(RD) \rightarrow c_{\text{initial}}. \quad (3)$$

A_{gen} 的初始代码生成工作流程如算法 1 所示。

算法 1. 初始代码生成。

输入: 需求文档 RD ;

输出: c_{initial} 。

① $c_0 \leftarrow \text{Generate}(RD)$; /*智能体根据需求文档调

用 LLM 生成第 1 版本代码*/

- ② for j in range (0, $maxcomple$) do /*执行编译反馈修正, $maxcomple$ 为最大编译反馈次数*/
- ③ $F_j \leftarrow A_{compile}(c_j)$; /*通过编译反馈智能体 $A_{compile}$ 执行编译检测来获取编译结果信息*/
- ④ If $\exists status \in F_j \wedge status = SUCCESS$ then
- ⑤ $c_{initial} \leftarrow c_j$; /*则返回编译完成的代码 c_j 作为 $c_{initial}$ */
- ⑥ break;
- ⑦ Else
- ⑧ $c_{j+1} \leftarrow Re\ Generate(c_j, F_j, RD)$; /*智能体根据编译反馈智能体 $A_{compile}$ 提供的信息修调再次用 LLM 修正代码编译错误*/
- ⑨ $c_{initial} \leftarrow c_{j+1}$;
- ⑩ End If
- ⑪ End for
- ⑫ Return $c_{initial}$ 。 /*输出结果*/

2.3 参考代码生成智能体

本文构建了一个参考代码生成智能体,旨在生成采用不同实现方式的高质量参考代码,让模型能够理解代码与需求间的映射,为后续的代码修正模型提供详细的修正建议。所生成的参考代码涵盖 3 种不同的实现风格:结构化编程风格、模块化编程风格和面向过程编程风格。这种方法背后的动机是,在软件开发中,开发人员在根据需求文档实现代码时可能有不同的理解,导致代码具有不同的实现风格。通过比较这些不同的代码实现,可以使模型能够更好地理解问题代码中的错误。

这 3 种实现风格之间的差异如表 1 所示。本文提出的框架采用 LLM 微调后作为参考代码生成智能体。

参考代码生成智能体 A_{ref} 的输入为用户提供的参考代码 c_{ref} 与其对应的需求文档 RD_{ref} , 输出为包含 3

种风格的代码集合 $C_{ref}\{stru, mod, pro\}$ 。 $stru$ 表示结构化(structural)编程风格的参考代码, mod 表示模块化(modular)编程风格的参考代码, pro 表示面向过程的(procedural)参考代码,形式化表达如式(4)所示。

$$A_{ref}(RD_{ref}, c_{ref}) \rightarrow C_{ref}。 \quad (4)$$

A_{ref} 生成参考代码的工作流程如算法 2 所示。

算法 2. 3 种风格的参考代码生成。

输入: 参考需求文档 RD_{ref} , 参考代码 c_{ref} ;

输出: C_{ref} 。

- ① $C_{ref} \leftarrow \{\}$; /*初始化风格参考代码集合*/
- ② $\{stru, mod, pro\} \leftarrow Generate(RD, c_{ref})$; /*智能体根据用户提供的参考代码与需求文档调用大模型生成初始风格参考代码集合*/
- ③ for i in $\{stru, mod, pro\}$ do /*开始对集合中的代码进行编译反馈操作*/
- ④ for j in range (0, $maxcomple$) do /*执行编译反馈修正, $maxcomple$ 为最大编译反馈次数*/
- ⑤ $F_j \leftarrow A_{compile}(i)$; /*通过编译反馈智能体 $A_{compile}$ 执行编译检测获取结构化编译结果信息*/
- ⑥ If $\exists status \in F_j \wedge status = SUCCESS$ then
/*如果编译结果通过,即状态 $status = SUCCESS$ */
- ⑦ $C_{ref} \leftarrow C_{ref} \cup \{i\}$; /*将编译通过的风格参考代码加入风格参考代码集合*/
- ⑧ break; /*跳出循环*/
- ⑨ Else /*编译未通过*/
- ⑩ $i \leftarrow Re\ Generate(i, F_j)$; /*智能体根据编译反馈智能体 $A_{compile}$ 提供的信息再次调用大模型修正代码编译错误*/
- ⑪ End If
- ⑫ End for
- ⑬ End for
- ⑭ Return C_{ref} 。 /*输出风格参考代码集合*/

2.4 代码优化建议智能体

本文构建了一个代码优化建议智能体用于对初始生成的代码进行修复工作,旨在通过学习 3 种风格的参考代码理解需求与代码的映射关系,理解代码与需求间的映射为后续的代码修正模型提供详细的修正建议。为了确保响应保持专注并维持高质量的代码修改,本文设计了一个基于推理的提示来构建该智能体,旨在引导模型审查参考代码和需求文档,然后识别问题代码中的错误位置并提供适当的

Table 1 Characteristics of Code Compiling with Different Styles

表 1 不同风格的代码编写特点

方面	结构化编程	模块化编程	面向过程编程
设计焦点	控制结构和程序逻辑	模块划分和接口设计	过程抽象和算法实现
代码组织	按控制结构组织(顺序、选择、循环)	按功能模块组织	按过程和函数组织
数据管理	局部变量为主,避免“go to”语句	模块封装数据+接口传递	函数参数、局部变量和返回值
接口设计	结构化控制流	明确的模块接口和契约	标准化函数接口

解决方案。

如图3所示,设计的提示包含3个组成部分:1)描述希望模型解决任务的说明(识别代码错误并提供解决方案);2)一小组示例(需求文档,参考代码)作为演示来帮助LLM理解和解决任务;3)由智能体生成解决方案。具体而言,我们构建了一个提示来引导模型通过理解参考代码和问题代码之间的不一致性来比较它们之间的差异,然后根据问题代码中的缺失元素提供相应的解决方案。为此阶段设计的提示如图4所示。

在智能体生成代码纠错策略后,智能体会将错误信息发送给需求驱动代码生成智能体处理已识别的问题并实施建议的修正,完成自动代码纠正过程,反馈交流过程如图5所示。在这些修正之后,需求驱动代码生成智能体会再次将代码发送给编译反馈智能体解决编译问题。该过程仅在代码没有任何“错误”警告时或达到最大编译纠正次数时结束。此时,从需求文档生成高质量代码的自动化过程基本完成。

代码优化建议智能体 A_{correct} 的输入为由需求驱动代码生成智能体 A_{gen} 生成的代码 c_{initial} 、需求文档 RD 、 RD_{ref} 、由参考代码生成智能体 A_{ref} 生成的参考代码集合 C_{ref} ,输出为格式化的参考修正建议 $suggestions$,在获得修正建议后进入初始代码修正流程,由需求驱动代码生成智能体 A_{gen} 根据生成编译验证后的最终代码 c_{output} 与由编译反馈智能体生成最终代码的可进行嵌入式系统测试的.hex文件。形式化表达如式(5)所示。

$$A_{\text{correct}}(RD, c_{\text{initial}}, RD_{\text{ref}}, C_{\text{ref}}) \rightarrow suggestions. \quad (5)$$

代码纠正工作流程如算法3所示。

算法3. 初始代码纠正。

输入: 需求文档 RD , 初始代码 c_{initial} , 需求参考文档 RD_{ref} , 参考代码集合 C_{ref} ;

输出: 最终代码 c_{output} , 最终代码的编译文件 hex 。

① $suggestions \leftarrow A_{\text{correct}}(RD, c_{\text{initial}}, RD_{\text{ref}}, C_{\text{ref}})$; /*代码优化建议智能体根据输入信息生成代码修正建议*/

```

您将担任一名经验丰富的嵌入式系统设计工程师。我将向您提供一份需求文档和对应的代码。请分析需求文档与参考代码之间的关系,
然后实现3个版本的代码。这些代码需保持与原始代码相同的功能逻辑, 但分别采用以下3种编程风格编写:
1. 结构化编程风格
2. 模块化编程风格
3. 面向过程编程风格。输出必须为完整的代码, 要求C语言代码。
输出要求:
1. 每个风格所有代码不分文件, 都采用一份文件实现。
2. 输出要求按照以下形式:
###结构化编程风格参考###“你的第1份代码”
###模块化编程风格参考###“你的第2份代码”
###面向过程编程风格参考###“你的第3份代码”。
3. 不要输出其他多余的东西
...

###需求文档:
需求文档:
简单电子时钟
系统概述:
该系统基于8051单片机(如STC89C52)设计。它利用DS1302实时时钟模块获取精确的时间数据, 并驱动16×2字符LCD显示屏实时显示
日期、时间和星期信息。该系统支持以下核心功能:
...

###参考代码1:
#include <reg52.h>
// Pin definitions
sbit DS1302_IO = P1^0;
...
// Global variables
unsigned char DateTime[7];
...
// Function declarations
void Init_LCD();
...
// Main function
void main(){...}
// Write byte to DS1302
void DS1302_WriteByte(unsigned char cmd, unsigned char dat) {...}
...

###参考代码:
...

```

Fig. 3 Key prompts for reference code generation agent

图3 参考代码生成智能体的关键提示词

您将扮演一名经验丰富的嵌入式系统设计工程师，负责指导错误代码的修正，并确保其完全满足项目需求。我会为您提供与当前项目类似的需求文档以及正确实现这些需求的参考代码。接下来，我会给出一段存在缺陷的代码及其对应的需求文档。请您按照以下步骤进行分析并提出修改建议：

1. 对比分析：阅读参考代码及其需求文档，识别错误代码中缺失或实现不当的功能点。
2. 制定修改策略：基于参考代码的正确实现，提出具体的代码修改策略，使错误代码能够达到需求要求，并与参考实现保持一致。
3. 输出规范：

代码修改策略： "..."
 解决方案： "..."

###需求文档:
 需求文档:
 简单电子时钟
 系统概述:
 该系统基于8051单片机（如STC89C52）设计。它利用DS1302实时时钟模块获取精确的时间数据，并驱动16×2字符LCD显示屏实时显示日期、时间和星期信息。该系统支持以下核心功能：
 ...

###参考代码:
 #include <reg52.h>
 // Pin definitions
 sbit DS1302_IO = P1^0;
 ...
 // Global variables
 unsigned char DateTime[7];
 ...
 // Function declarations
 void Init_LCD();...
 // Main function
 void main(){...}
 // Write byte to DS1302
 void DS1302_WriteByte(unsigned char cmd, unsigned char dat) {...}

Fig. 4 Key prompts for code optimization suggestion agent

图4 代码优化建议智能体关键提示词

您提供的代码存在以下问题。请按照给定的纠正策略系统地解决这个问题，并最终提供完整的最终纠正代码。

###代码修正策略:
 “错误代码存在多个问题，包括DS1302通信协议不正确、缺少BCD码转换、缺乏LCD忙检测、格式化显示不符合要求，以及报警功能实现不当。
 需要基于参考代码纠正这些问题，以确保系统正确读取时间数据并按照规定要求进行显示。”

###解决方案:
 1. DS1302通讯纠正:
 DS1302_ReadByte 函数不正确。...
 4. 其他问题: ...
 具体修正:
 1. 修正 DS1302 reading 函数:
 (code):...

Fig. 5 Communication content sent by code correction agent to requirement-driven code generation agent

图5 代码纠正智能体发送给需求驱动代码生成智能体的交流内容

- ② $output_0 \leftarrow A_{gen}(suggestion, c_{initial})$; /*需求驱动代码生成智能体根据修正建议生成修正后的代码*/
- ③ for j in range $(0, maxcomple)$ do /*执行编译反馈修正, $maxcomple$ 为最大编译反馈次数*/
- ④ $F_j \leftarrow A_{compile}(output_0)$; /*通过编译反馈智能体 $A_{compile}$ 执行编译检测获取结构化编译结果信息*/
- ⑤ If $\exists status \in F_j \wedge status = SUCCESS$ then
- ⑥ $c_{output} \leftarrow output_j$; /*将编译通过的最终代码

保存*/

- ⑦ $hex \leftarrow A_{compile}(c_{output})$; /*编译智能体生成最终代码的.hex 编译文件*/
- ⑧ break;
- ⑨ Else
- ⑩ $output_{i+1} \leftarrow Re\ Generate(F_j, output_i)$; /*智能体根据编译反馈智能体 $A_{compile}$ 提供的信息调用大模型修正代码编译错误*/
- ⑪ End If
- ⑫ End for
- ⑬ Return c_{output}, hex . /*输出风格参考代码集合*/

3 实验分析

为了评估所提出方法的有效性，本文进行了初步实验。本节描述了实验设计，包括研究问题、模型规格、基准测试、评估指标和实验细节。我们通过解决以下4个研究问题来评估所提出的模型。

RQ1: 与基线方法相比，本文提出的方法在嵌入式系统代码生成质量上如何？

RQ2: 本文提出的方法在不同的LLM上的表现如何？

RQ3: 多智能体协作架构是否能够提升代码生成质量？

RQ4: 对参考代码的依赖情况如何?

RQ5: 本文提出的方法能否缩短嵌入式系统代码开发时间?

3.1 测试基准

在本研究中,我们选择了 Claude 3.7 Sonnet, Claude 4.0 Sonnet, GPT4.1, GPT4o 作为智能体的基础模型与对比基准,所有这些模型的调用均采用 API 形式进行。同时我们选择了现有的较为先进的基于 LLM 的代码生成方案作为对比基准。

SCOT^[4]方法则通过结构化 CoT 提示,在生成代码之前,引导 LLM 以伪代码形式梳理和理解代码生成所需的中间步骤。

ROCODE^[26]方法集成了回溯机制和程序分析技术,通过在 LLM 代码生成过程中引入回溯搜索策略,当生成的代码片段不满足程序分析约束时能够自动回退并尝试替代方案,从而提高代码生成的正确性和可靠性。

MBR-EXEC^[27]方法基于执行结果的最小贝叶斯风险框架,通过生成多个候选代码解决方案并实际执行测试用例,利用执行结果来评估和选择最优的代码版本,有效提升了自然语言到代码翻译的准确性。

MGD^[28]方法采用监控器引导解码策略,通过静态分析监控器在解码过程中实时检查类型一致性和对象解引用等约束条件,为 LLM 提供全局上下文信息,减少代码生成中的幻觉现象并提高生成代码的质量。

ChatDev^[41]方法构建了一个基于聊天链的多智能体软件开发框架,通过将软件开发过程分解为设计、编码和测试等阶段,使用专业化的智能体(如 CEO、CTO、程序员、测试员等)进行多轮对话协作,实现从需求到完整软件的自动化开发,并通过交流去幻觉机制减少代码生成中的错误。

AgentCoder^[40]方法提出了一个精简的三智能体协作框架,包括程序员智能体、测试设计师智能体和测试执行器智能体,通过独立的测试用例生成和执行反馈机制,实现高效的代码生成和迭代优化,相比现有多智能体方法显著降低了 token 开销的同时提升了代码生成质量。

Codex^[2]是 OpenAI 开发的专门针对代码生成任务的 LLM,基于 GPT 架构并在大规模代码数据集上进行训练。Codex 能够理解自然语言描述并生成相应的代码,支持多种编程语言,在代码补全、函数生成和程序合成等任务上表现出色,是 GitHub Copilot 等代码辅助工具的核心技术基础。

Cursor^[41]是一款基于 LLM 的智能代码编辑器,集成了 GPT-4 等先进模型,提供实时代码生成、智能补全和代码重构等功能。Cursor 通过上下文感知的代码理解能力,能够根据用户的自然语言描述或代码注释自动生成相应的代码片段,并支持多轮对话式的代码开发交互。

3.2 实验数据

鉴于缺乏针对小型嵌入式系统的测试数据集,本文构建了一个包含 20 个小型嵌入式系统模拟实现的数据集,现已公开发布 RD2ESCODE1(GitHub-Hongbin-Xiao/RD2ESCODE)。该数据集涵盖了从简单到复杂的 20 个小型嵌入式系统需求文档,既包括可用于生成需求文档的参考代码,也包含 3 种不同风格的代码实现作为模型参考。此外,每个案例还配套有可在 Proteus 8 Professional 中运行的可执行仿真系统。数据集中最简单的系统仅包含 1 个函数,而最复杂的系统约有 900 行代码。

3.3 实验数据

本文采用 Pass@ k ^[5]指标评估生成代码的准确性,该指标已在以往与 LLM 相关的研究中得到广泛应用。Pass@ k 指标的含义是:只要生成的多个代码解决方案中有任意一个通过了全部测试,即认为该问题被成功解决。然而,在实际开发场景中,生成多个代码版本会增加开发人员的负担,因为他们需要阅读、理解并从中选择最终的目标代码。基于此考虑,本研究将 k 设置为 1,以更贴合大多数开发者仅关注单一生成代码的实际情况。为降低评测结果的高方差和随机性,每种方法均重复运行 3 次,并以平均结果作为最终报告。此外,为了全面衡量生成代码的质量,我们采用以下 2 个指标进行评估:

1) 代码编译通过率: 3 次运行中代码成功编译的平均比例。

2) 平均测试功能通过率: 3 次运行中每个系统功能测试通过的平均比例。(关于测试程序的详细说明,可参见数据集中提供的 Test Procedure Reference.xlsx 文件。)

3.4 实验数据

本实验在一台配备英特尔第 13 代酷睿处理器(Intel Core i7-13400)的计算机上进行。该处理器拥有 10 个物理核心(6 个性能核+4 个能效核)、16 线程,基础频率为 2.5 GHz。计算机配备 32 GB 内存,显卡为 NVIDIA GeForce RTX 3070 Ti。操作系统为 Windows 11 专业版,64 位架构,版本号 22H2。实验采用 Python 编程语言,版本为 Python3.10,开发和运行

环境使用 PyCharm 集成开发环境 (IDE), 并利用 Anaconda3 进行环境管理与依赖配置, 以确保实验环境的隔离性与可重复性。

3.5 实验结果分析

本研究采用标准的嵌入式系统代码生成评估框架进行实验验证。实验选取了具有代表性的基线方法, 包括主流 LLM Claude 3.7 Sonnet、Claude 4 Sonnet、GPT-4o 和 GPT-4.1, 以及先进的代码生成方法 SCOT、MBR-EXEC、MGD 和 ROCODE 作为对比基准, 为了保证公平, 基础大模型均采用其自反思 6 次后的结果。评估采用 3 个核心指标: 1) Pass@1 衡量首次生成代码的功能正确性; 2) 平均测试功能通过率评估代码在完整测试上的表现; 3) 平均代码编译通过率反映生成代码的语法正确性。为了全面评估方法的稳定性和适应性, 实验采用 Claude 4 Sonnet 作为本文方法的智能体基础模型, 在多个温度参数 T 设置 ($T=0, 0.4, 0.6, 0.8$) 下进行测试, 其中 $T=0$ 表示确定性生成, $T=0.4$ 表示默认平衡设置, 更高温度值引入更多随机性以探索生成多样性的影响。所有实验在相同的硬件环境和数据集上进行, 确保结果的公平性和可比性。

RQ1: 与基线方法相比, 本文提出的方法在嵌入式系统代码生成质量上如何?

基于表 2 的实验结果, RD2ESC 方法在嵌入式系统代码生成质量上实现了显著突破。在 Pass@1 指标上, RD2ESC 在 $T=0.4$ 时达到 71.67% 的最佳表现, 相比最强基线 ROCODE(55%) 提升 16 个百分点, 相比传统方法 Claude 4 Sonnet(18.33%) 和 GPT-4o(15%), 这种性能差距揭示了单一模型在处理复杂嵌入式系统约束时的局限性。在平均测试功能通过率方面, RD2ESC 最高达到 75%, 比 ROCODE(68%) 提升 7 个百分点, 比同类多智能体方法 ChatDev(48%) 和 Agent-Coder(55%) 分别提升 27 个百分点和 25 个百分点, 表明专门化的多智能体系统在理解和实现复杂功能需求方面具有显著优势。在平均代码编译通过率上, RD2ESC 表现尤为突出, 在 $T=0$ 和 $T=0.4$ 时均达到 95% 的峰值, 与传统方法的 38%~65% 形成鲜明对比, 体现了其编译反馈智能体和迭代优化机制的有效性。温度敏感性分析显示, RD2ESC 在 $T=0.4$ 时达到功能正确性与编译成功率的最佳平衡, 随着温度升高, 性能逐步下降但仍优于所有基线方法, 展现出优秀的鲁棒性和稳定性。

RQ2: 本文提出的方法在不同的 LLM 上的表现如何?

Table 2 Comparative Results of Code Generation Quality and Baseline Models of RD2ESC

表 2 RD2ESC 的代码生成质量与基准模型对比结果 %

方法	Pass@1	平均测试功能 通过率	平均代码编译 通过率
Claude 3.7 Sonnet	13.33	35	41
Claude 4 Sonnet	18.33	41	48
GPT-4o	15	35	40
GPT-4.1	16.67	35	38
Codex	13.33	30	28
Cursor	18.33	40	46
SCOT	20	33	46
MBR-EXEC	21.67	45	55
MGD	28.33	48	65
ROCODE	55	68	93
ChatDev	35.00	48	53
AgentCoder	48.33	55	85
RD2ESC ($T=0$)	68.33	73	95
RD2ESC ($T=0.4$)	71.67	75	95
RD2ESC ($T=0.6$)	68.33	71	91
RD2ESC ($T=0.8$)	61.67	68	88

在本研究问题中, 我们旨在探究所提出的方法在不同的 LLM 作为智能体的基础上表现如何, 我们采用最优温度在 Claude 3.7 Sonnet、Claude 4 Sonnet、GPT-4o 和 GPT-4.1 上对数据进行了测试, 温度设置均为 0.4。

如图 6 所示, RD2ESC 方法在不同 LLM 上均展现出卓越的适应性和稳定性, 验证了其多智能体架构的通用性。Claude 4 Sonnet 作为底层模型时表现最佳, Pass@1 达到 71%, 平均测试功能通过率达到 75%, 相比 Claude 3.7 Sonnet 分别提升 3 个百分点和 2 个百分点, 反映了新一代模型更强的代码理解能力。GPT 系列模型表现相对稳定, GPT-4.1 (Pass@1=68%)

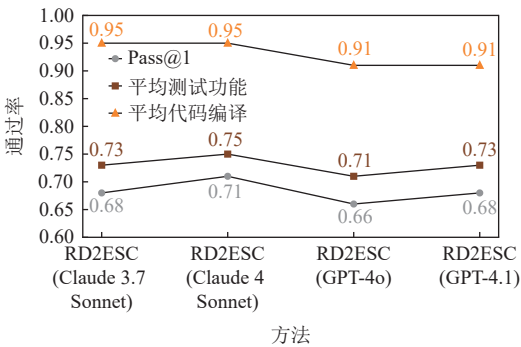


Fig. 6 Performance comparison of different models on RD2ESC

图 6 不同模型在 RD2ESC 上的性能对比

略低于 GPT-4o(66%), 但差异较小, 说明 RD2ESC 能够有效弥合不同模型间的性能差异。值得注意的是, 所有模型在 RD2ESC 框架下的编译通过率均保持在 91%~95% 的高水平, 证明了多智能体协作机制在语法正确性保障方面的有效性。更重要的是, 即使是相对较弱的 GPT-4o 在 RD2ESC 框架下仍能达到 66% 的 Pass@1 表现, 远超其单独使用时的 15%, 这表明 RD2ESC 的性能提升主要源于其创新的多智能体协作机制而非单纯依赖底层模型能力, 为该方法在不同技术上的广泛应用提供了有力支撑。

RQ3: 多智能体协作架构是否能够提升代码生成质量?

在本研究问题中, 我们旨在探讨所设计的关键模型结构以及编译反馈机制的必要性。首先, 在原有模型的基础上移除代码优化建议智能体, 得到 RD2ESC-NT(无纠正智能体); 随后, 在原有模型基础上移除编译反馈智能体, 得到 RD2ESC-NF(无编译反馈); 最后移除参考代码生成智能体(RD2ESC-NSC)。同样在该数据集上进行测试。此处 Pass@ k 中的 k 仍取 1, 每个案例的代码生成次数依然为 3。

从图 7 中可以看出 RD2ESC 框架中智能体协作的非线性协同效应和功能分工的关键性。通过系统性移除核心组件, 我们观察到代码优化建议智能体的移除导致 Pass@1 性能出现灾难性下降(从 71% 降至 16%, 相对下降约 55 个百分点), 而编译反馈智能体的移除造成显著性能衰减(降至 58%, 相对下降约 13 个百分点), 参考代码生成智能体的移除导致性能下降至 63%(相对下降约 8 个百分点)。这种差异化衰减模式表明: 1) 代码优化建议智能体承担着将抽象错误诊断转化为具体修复操作的关键功能, 是系

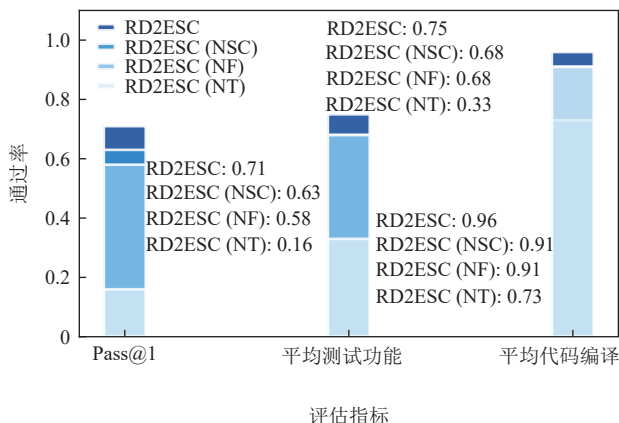


Fig. 7 Performance comparison results of RD2ESC under different structures

图 7 不同结构下的 RD2ESC 性能对比结果

统性能的主要决定因素; 2) 编译反馈智能体通过提供精确的错误定位和语义分析, 为纠错过程提供了必要的指导信息, 其移除导致代码编译通过率从 96% 骤降至 73%; 3) 参考代码生成智能体通过提供多样化代码模板, 为系统提供了有效的结构化参考。更重要的是, 完整系统相较于任何单一组件都展现出显著的协同放大效应, 证实了多智能体协作结构在代码生成任务中的设计合理性。

RQ4 对参考代码的依赖情况如何?

为了评估本文提出的方法对参考代码的依赖情况, 我们设计了参考代码质量敏感性分析实验。实验通过对原始参考代码引入不同程度的扰动来模拟实际开发中可能遇到的参考代码质量变化, 具体设置 4 个测试场景: 高质量参考代码(10% 扰动)、中等质量参考代码(30% 扰动)、低质量参考代码(50% 扰动)以及完全无参考代码。扰动操作涵盖嵌入式开发中的 6 个关键维度: 参数调整(传感器阈值、延时参数等数值修改)、变量和函数重命名、引脚重新分配、算法优化(添加滤波和错误处理机制)、功能扩展(增加监控和诊断功能)以及代码结构重构。通过系统地降低参考代码质量, 该实验旨在量化方法性能随参考代码可用性和质量变化的衰减程度, 从而客观评估方法在面对新领域或特殊硬件平台时参考代码稀缺情况下的实用边界, 为方法的实际应用提供重要的性能预期指导。

从图 8 中可以看出, 随着参考代码质量的下降, 方法性能呈现明显的递减趋势: Pass@1 从 10% 扰动时的 0.68 逐步下降至 30% 扰动的 0.61 及 50% 扰动的 0.46, 在完全无参考代码时大幅降至 0.25, 累计性能损失达 63%; 测试通过率和编译通过率也表现出类似的下降模式, 分别从 0.73 和 0.96 降至 0.38 和 0.65。

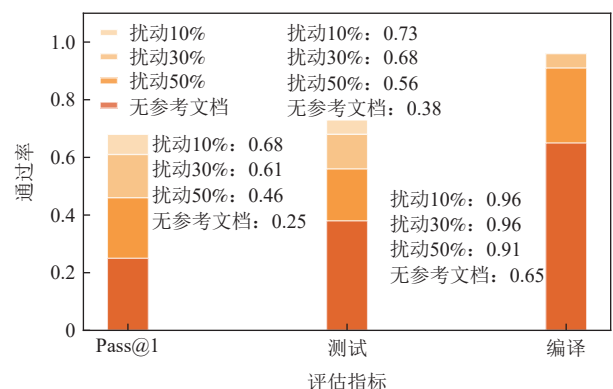


Fig. 8 Evaluation of dependency of RD2ESC on reference code

图 8 RD2ESC 对参考代码的依赖性评估

值得注意的是,编译通过率在中等扰动下保持相对稳定(10%, 30% 扰动时均为 0.96),说明生成代码的语法正确性具有一定鲁棒性,主要性能损失集中在逻辑实现的准确性上。这一结果表明我们的方法确实对参考代码质量存在依赖性,特别是在缺乏参考代码的新领域或特殊硬件平台上性能会显著下降,但即使在极端条件下仍保持基础的代码生成能力。

RQ5: 本文提出的方法能否缩短嵌入式系统代码开发时间?

为了评估本文方法能否帮助人们提高开发效率,我们开发了一款工具—RD2ESCODE^①。同时我们开展了一项实验,邀请 6 名研究生(A-F)分别独立完成 15 个嵌入式系统项目。我们评估了 3 种不同的开发方式:手动开发、使用高质量参考代码开发、借助传统大模型开发以及借助 RD2ESCODE 工具辅助开发。对于每种方法,我们记录了每位学生完成所有项目所需的时间,测量从收到需求文档到代码成功编译的全过程时长。需要注意的是,这一测量不包括后续的功能测试或验证。

如表 3 所示,实验结果表明 RD2ESCODE 在开发时间缩短方面表现卓越,将平均开发时间从手动开发的 29.3 h 大幅缩短至 4.1 h,实现了 86.0% 的时间节省和超过 7 倍的效率提升;相比借助参考代码开发的 13.2 h, RD2ESCODE 仍实现了 69.0% 的时间节省和约 3.2 倍的效率提升。更重要的是, RD2ESCODE 在保持高效率的同时确保了代码质量,其相对手动开发的提升比例平均达到 85.7%,相对参考代码提升比例平均达到 68.2%,所有参与者的开发时间均稳定控制在 3~5 h,展现出工具的稳定性和可靠性。值得注意的是,6 名参与者在使用 RD2ESCODE 时均表现出一致的高效率,相对手动开发的提升比例稳定在 83%~88% 之间,相对参考代码的提升比例稳定在 62%~74% 之间,显示出方法的普适性。这些结果充分验证了 RD2ESC 通过多智能体协作机制实现了效率与质量的双重优化,为嵌入式系统需求到代码的自动生成提供了实用的智能化解决方案。

3.6 有效威胁性

上述实验结果充分验证了 RD2ESCODE 的有效性,然而为了客观评估方法的适用范围,我们需要进一步讨论其在实际部署中可能面临的挑战和局限性。

1) 输入长度限制。我们的方法需要在 LLM 输入中保留完整的对话历史、参考代码和需求文档,这对

Table 3 Comparative Results of Development Time under Different Methods

表 3 在不同方法下的开发时间对比结果

学生	手动开发	借助参考 代码开发	借助大模 型开发	借助工 具开发	相对手 工提升 比例/%	相对参考 代码提升 比例/%
A	22 h	9.5 h	12 h	3 h 10 min	86	67
B	27.5 h	12 h	15 h	3 h 45 min	86	69
C	34 h	15.5 h	18 h	4 h 05 min	88	74
D	29.5 h	13.5 h	16 h	4 h 55 min	83	64
E	38 h	17 h	20 h	4 h 40 min	88	73
F	25 h	11.5 h	13 h	4 h 20 min	83	62

于大型嵌入式系统来说可能超出模型的上下文窗口限制(如 Claude 的 200 000 tokens)。这一限制在处理大型复杂的嵌入式系统中较为明显,未来工作将探索分层代码生成策略,将大型系统分解为多个子模块分别处理,或采用检索增强生成技术动态加载相关代码片段。

2) 对参考代码的依赖。当前方法依赖高质量的参考代码来建立需求与实现之间的映射关系。在缺乏相似参考代码的新领域或特殊硬件平台上方法性能会有所下降。

3) 数据规模限制。我们的评估基于 20 个精心设计的测试案例,这些案例覆盖了嵌入式系统开发中的核心场景:传感器数据采集、通信协议实现、实时控制逻辑和数据处理算法。每个案例都包含了实际项目中的典型需求模式和编程挑战。虽然这一数据集规模在统计意义上存在一定局限性,但考虑到嵌入式系统代码生成任务的复杂性和每个案例的综合性,当前的评估基本能有效展示方法的核心能力。

4 总 结

将需求文档自动转化为嵌入式系统代码是一个极具挑战性的目标。我们的初步实验发现,在 LLM 理解和生成代码的过程中,通过多智能体协作结构能够帮助模型理解需求文档与代码之间的关系,并引导其对生成代码进行修正,能够有效提升其在嵌入式代码生成方面的能力。这一过程有望实现从需求文档到小型嵌入式系统的无缝衔接。然而,现有的研究仍处于初步阶段,未来还有许多方向值得进一步探索,以提升模型的整体能力。基于我们的初步结果,后续工作有望成为实现从需求到嵌入式代

① 工具已经开源,演示视频链接: <https://youtu.be/5dz5B-s-TAU>, 开源链接: <https://github.com/Hongbin-Xiao/RD2ESCODE-Tool>

码自动生成的重要一步,并为利用需求到代码映射方法提升LLM生成系统级嵌入式代码的研究提供新的方向。

作者贡献声明: 谭舒孺负责方案实验整体设计并撰写论文;肖宏彬、武天昊负责算法思路和实验方法;李智、谢晓兰、汤飞提出指导意见并修改论文。李智和谢晓兰为通信作者;谭舒孺和肖宏彬对本文具有相同贡献。

参 考 文 献

- [1] Moreira T G, Wehrmeister M A, Pereira C E, et al. Automatic code generation for embedded systems: From UML specifications to VHDL code[C]//Proc of the 2010 8th IEEE Int Conf on Industrial Informatics. Piscataway, NJ: IEEE, 2010: 1085–1090
- [2] Chen Mark, Tworek Jerry, Jun Heewoo, et al. Evaluating large language models trained on code[J]. arXiv preprint, arXiv: 2107.03374, 2021
- [3] Li Jia, Zhao Yunfei, Li Yongmin, et al. Acecoder: An effective prompting technique specialized in code generation[J]. *ACM Transactions on Software Engineering and Methodology*, 2024, 33(8): 1–26
- [4] Li Jia, Li Ge, Li Yongmin, et al. Structured chain-of-thought prompting for code generation[J]. *ACM Transactions on Software Engineering and Methodology*, 2025, 34(2): 1–23
- [5] Bambagini M, Di N M. A code generation framework for distributed real-time embedded systems[C]//Proc of the 2012 IEEE 17th Int Conf on Emerging Technologies & Factory Automation. Piscataway, NJ: IEEE, 2012: 1–10
- [6] Vidal J, De Lamotte F, Gogniat G, et al. A co-design approach for embedded system modeling and code generation with UML and MARTE[C]//Proc of the 2009 Design, Automation & Test in Europe Conf & Exhibition. Piscataway, NJ: IEEE, 2009: 226–231
- [7] Staron M, Durisic D. Autosar Standard[M]//Automotive Software Architectures: An Introduction. Berlin: Springer, 2017: 81–116
- [8] Dunne M, Schram K, Fischmeister S. Weaknesses in LLM-generated code for embedded systems networking[C]//Proc of the 2024 IEEE 24th Int Conf on Software Quality, Reliability and Security. Piscataway, NJ: IEEE, 2024: 250–261
- [9] Dong Yihong, Li Ge, Jin Zhi. CODEP: Grammatical seq2seq model for general-purpose code generation[C]//Proc of the ISSTA. New York: ACM, 2023: 188–198
- [10] Rozière B, Gehring J, Gloeckle F, et al. Code Llama: Open foundation models for code[J]. arXiv preprint, arXiv: 2308.12950, 2023
- [11] Nijkamp E, Pang Bo, Hayashi H, et al. Codegen: An open large language model for code with multi-turn program synthesis[C]//Proc of the ICLR. OpenReview. net, 2023: 1–25
- [12] Fried D, Aghajanyan A, Lin J, et al. Incoder: A generative model for code infilling and synthesis[J]. arXiv preprint, arXiv: 2204.05999, 2022
- [13] Jiang Xue, Dong Yihong, Wang L, et al. Self-planning code generation with large language models[J]. *ACM Transactions on Software Engineering and Methodology*, 2024, 33(7): 1–36
- [14] Jiang Xue, Dong Yihong, Jin Zhi, et al. SEED: Customize large language models with sample-efficient adaptation for code generation[J]. arXiv preprint, arXiv: 2403.00046, 2024
- [15] Zhang Tianyi, Yu Tao, Hashimoto T, et al. Coder reviewer reranking for code generation[C]//Proc of the ICML. New York: PMLR, 2023: 41832–41846
- [16] GitHub. Copilot[EB/OL]. 2022[2024-12-25]. <https://github.com/features/copilot>
- [17] Dohmke T, Iansiti Ma, Richards G. Sea change in software development: Economic and productivity analysis of the Ai-powered developer lifecycle[J]. arXiv preprint, arXiv: 2306.15033, 2023
- [18] Ahmad W U, Chakraborty S, Ray B, et al. Unified pre-training for program understanding and generation[C]//Proc of the NAACL-HLT. Stroudsburg, PA: ACL, 2021: 2655–2668
- [19] Wang Yue, Wang Weishi, Joty S R, et al. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation[C]//Proc of the 2021 Conf on Empirical Methods in Natural Language Processing. Stroudsburg, PA: ACL, 2021: 8696–8708
- [20] Li YuJia, Choi D, Chung J Y, et al. Competition level code generation with AlphaCode[J]. arXiv preprint, arXiv: 2203.07814, 2022
- [21] Black S, Biderman S, Hallahan E, et al. GPT-NeoX-20B: An open-source autoregressive language model[J]. arXiv preprint, arXiv: 2204.06745, 2022
- [22] Xu F F, Alon U, Neubig G, et al. A systematic evaluation of large language models of code[C]//Proc of the 6th ACM SIGPLAN Int Symp on Machine Programming. New York: ACM, 2022: 1–10
- [23] OpenAI. ChatGPT[EB/OL]. 2022. [2024-11-17]. <https://openai.com/blog/chatgpt>
- [24] OpenAI. GPT-4 technical report[J]. arXiv preprint, arXiv: 2303.08774, 2023
- [25] Jiang Xue, Dong Yihong, Tao Yingwei, et al. Rocode: Integrating backtracking mechanism and program analysis in large language models for code generation[J]. arXiv preprint, arXiv: 2411.07112, 2024
- [26] Shi F, Fried D, Ghazvininejad M, et al. Natural language to code translation with execution[J]. arXiv preprint, arXiv: 2204.11454, 2022
- [27] Agrawal L A, Kanade A, Goyal N, et al. Guiding language models of code with global context using monitors[J]. arXiv preprint, arXiv: 2306.10763, 2023
- [28] Brown T B, Mann B, Ryder N, et al. Language models are few-shot learners[C]//Proc of the 34th Annual Conf on Neural Information Processing Systems. Red Hook, NY: Curran Associates, 2020: 1877–1901
- [29] Dong Yihong, Jiang Xue, Jin Zhi, et al. Self-collaboration code generation via ChatGPT[J]. arXiv preprint, arXiv: 2304.07590, 2023
- [30] Liu Chao, Bao XuanLin, Zhang Hongyu, et al. Improving ChatGPT prompt for code generation[J]. arXiv preprint, arXiv: 2305.08360, 2023
- [31] Nashid N, Sintaha M, Mesbah A. Retrieval-based prompt selection for

code-related few-shot learning[C]//Proc of the 45th IEEE/ACM Int Conf on Software Engineering. Piscataway, NJ: IEEE, 2023: 2450–2462

- [32] Shrivastava D, Larochelle H, Tarlow D. Repository-level prompt generation for large language models of code[J]. arXiv preprint, arXiv: 2206.12839, 2022
- [33] Wei J, Wang Xuezhi, Schuurmans D, et al. Chain of thought prompting elicits reasoning in large language models[J]. arXiv preprint, arXiv: 2201.11903, 2022
- [34] Kojima T, Gu S S, Reid M, et al. Large language models are zero-shot reasoners[J]. arXiv preprint, arXiv: 2205.11916, 2022
- [35] Li Jia, Li Ge, Li Yongmin, et al. Enabling programming thinking in large language models toward code generation[J]. arXiv preprint, arXiv: 2305.06599, 2023
- [36] Wang Chaozheng, Yang Yuanhang, Gao Cuiyun, et al. Prompt tuning in code intelligence: An experimental evaluation[J]. *IEEE Trans on Software Engineering*, 2023, 49(11): 4869–4885
- [37] Mu Fangwen, Shi Lin, Wang Shuai, et al. Clarifygpt: A framework for enhancing LLM-based code generation via requirements clarification[J]. *Proceedings of the ACM on Software Engineering*, 2024, 1(FSE): 2332–2354
- [38] Liu Jiawei, Xia C S, Wang Yuyao, et al. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation[J]. arXiv preprint, arXiv: 2305.01210, 2023
- [39] Chen Qian, Liu Wei, Liu Hongzhang, et al. ChatDev: Communicative agents for software development[C]//Proc of the 2024 Conf on Empirical Methods in Natural Language Processing. Stroudsburg, PA: ACL, 2024: 1–15
- [40] Huang Dong, Zhang J M, Luck M, et al. AgentCoder: Multi-agent code generation with effective testing and self-optimisation[J]. arXiv preprint, arXiv: 2312.13010, 2024
- [41] Anysphere Inc. Cursor Documentation[EB/OL]. [2024-11-17]. <https://cursor.com/docs>



Tan Shuru, born in 1998. PhD candidate. Student member of CCF. His main research interests include intelligent code generation, intelligent information processing, and requirements engineering.

谭舒孺, 1998年生。博士研究生。CCF学生会员。主要研究方向为智能代码生成、智能信息处理、需求工程。



Xiao Hongbin, born in 1997. PhD candidate. Student member of CCF. His main research interests include requirements engineering, prompt engineering, and software engineering.

肖宏彬, 1997年生, 博士研究生。CCF学生会员。主要研究方向为需求工程、提示工程、软件工程。



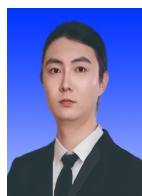
Li Zhi, born in 1969. PhD, professor, PhD supervisor. Distinguished member of CCF. His main research interests include software engineering and requirements engineering.

李智, 1969年生。博士, 教授, 博士生导师。CCF杰出会员。主要研究方向为软件工程、需求工程。



Xie Xiaolan, born in 1974. PhD, professor, PhD supervisor. Her main research interests include big data, cloud computing, and manufacturing informatization.

谢晓兰, 1974年生。博士, 教授, 博士生导师。主要研究方向为大数据、云计算、制造业信息化。



Wu Tianhao, born in 1998. Master. His main research interests include machine learning, graph neural networks, and embedded code generation.

武天昊, 1998年生。硕士。主要研究方向为机器学习、图神经网络、嵌入式代码生成。



Tang Fei, born in 1978. Bachelor. Senior member of CCF, and a TOGAF enterprise architect. His main research interests include enterprise architecture, requirements engineering, and prompt engineering.

汤飞, 1978年生。学士。CCF高级会员, TOGAF企业架构师。主要研究方向为企业架构、需求工程、提示工程。